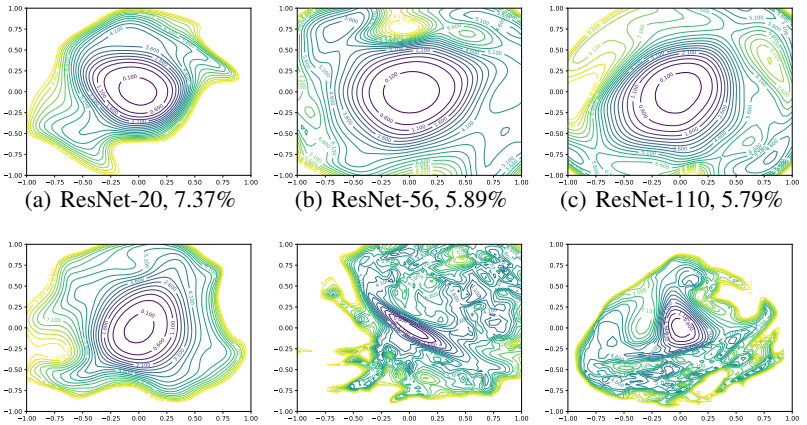


(a) ResNet-20, 7.37% (b) ResNet-56, 5.89% (c) ResNet-110, 5.79%

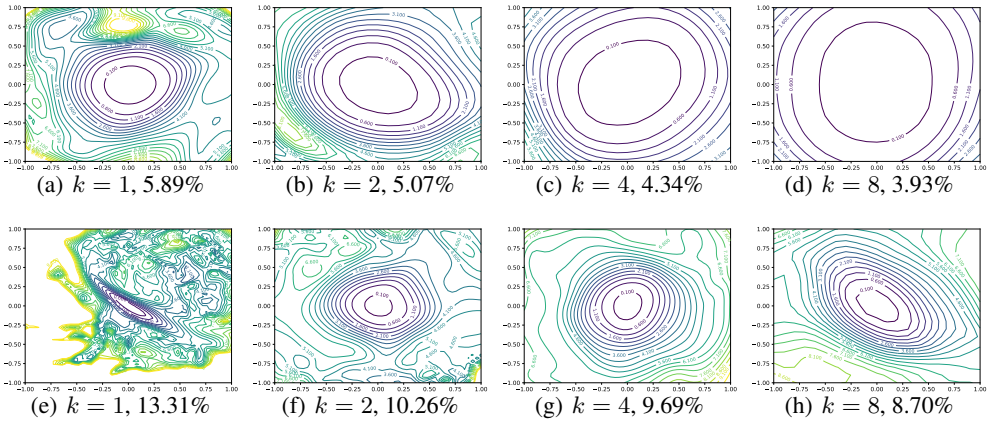
(d) ResNet-20-NS, 8.18% (e) ResNet-56-NS, 13.31% (f) ResNet-110-NS, 16.44%

Figure 5: 2D visualization of the loss surface of ResNet and ResNet-noshort with different depth.

Experimental Setup To understand the effects of network architecture on non-convexity, we trained a number of networks, and plotted the landscape around the obtained minimizers using the filter-normalized random direction method described in Section 4. We consider three classes of neural networks: 1) ResNets [17] that are optimized for performance on CIFAR-10. We consider ResNet-20/56/110, where each name is labeled with the number of layers it has. 2) "VGG-like" networks that do not contain shortcut/skip connections. We produced these networks simply by removing the shortcut connections from ResNets. We call these networks ResNet-20/56/110-noshort. 3) "Wide" ResNets that have more filters per layer than the CIFAR-10 optimized networks. All models are trained on the CIFAR-10 dataset using SGD with Nesterov momentum, batch-size 128, and 0.0005 weight decay for 300 epochs. The learning rate was initialized at 0.1, and decreased by a factor of 10 at epochs 150, 225 and 275. Deeper experimental VGG-like networks (e.g., ResNet-56-noshort, as described below) required a smaller initial learning rate of 0.01. High resolution 2D plots of the minimizers for different neural networks are shown in Figure 5 and Figure 6. Results are shown as contour plots rather than surface plots because this makes it extremely easy to see non-convex structures and evaluate sharpness. For surface plots of ResNet-56, see Figure 1. Note that the center of each plot corresponds to the minimizer, and the two axes parameterize two random directions with filter-wise normalization as in (1). We make several observations below about how architecture affects the loss landscape.

The Effect of Network Depth From Figure 5, we see that network depth has a dramatic effect on the loss surfaces of neural networks when skip connections are not used. The network ResNet-20-noshort has a fairly benign landscape dominated by a region with convex contours in the center, and no dramatic non-convexity. This isn't too surprising: the original VGG networks for ImageNet had 19 layers and could be trained effectively [33]. However, as network depth increases, the loss surface of the VGG-like nets spontaneously transitions from (nearly) convex to chaotic. ResNet-56-noshort has dramatic non-convexities and large regions where the gradient directions (which are normal to the contours depicted in the plots) do not point towards the minimizer at the center. Also, the loss function becomes extremely large as we move in some directions. ResNet-110-noshort displays even more dramatic non-convexities, and becomes extremely steep as we move in all directions shown in the plot. Furthermore, note that the minimizers at the center of the deep VGG-like nets seem to be fairly sharp. In the case of ResNet-56-noshort, the minimizer is also fairly ill-conditioned, as the contours near the minimizer have significant eccentricity.

Shortcut Connections to the Rescue Shortcut connections have a dramatic effect of the geometry of the loss functions. In Figure 5, we see that residual connections prevent the transition to chaotic behavior as depth increases. In fact, the width and shape of the 0.1-level contour is almost identical for the 20- and 110-layer networks. Interestingly, the effect of skip connections seems to be most important for deep networks. For the more shallow networks (ResNet-20 and ResNet-20-noshort), the effect of skip connections is fairly unnoticeable. However residual connections prevent the explosion of non-convexity that occurs when networks get deep. This effect seems to apply to other kinds



(a) $k = 1$, 5.89% (b) $k = 2$, 5.07% (c) $k = 4$, 4.34% (d) $k = 8$, 3.93%

(e) $k = 1$, 13.31% (f) $k = 2$, 10.26% (g) $k = 4$, 9.69% (h) $k = 8$, 8.70%

Figure 6: Wide-ResNet-56 on CIFAR-10 both with shortcut connections (top) and without (bottom). The label $k = 2$ means twice as many filters per layer. Test error is reported below each figure.

of skip connections as well; Figure 4 show the loss landscape of DenseNet [20], which shows no noticeable non-convexity.

Wide Models vs Thin Models To see the effect of the number of convolutional filters per layer, we compare the narrow CIFAR-optimized ResNets (ResNet-56) with Wide-ResNets [41] by multiplying the number of filters per layer by $k = 2, 4$, and 8. From Figure 6, we see that the wider models have loss landscapes with no noticeable chaotic behavior. Increased network width resulted in flat minima and wide regions of apparent convexity. We see that increased width prevents chaotic behavior, and skip connections dramatically widen minimizers. Finally, note that sharpness correlates extremely well with test error.

Implications for Network Initialization One interesting property seen in Figure 5 is that loss landscapes for all the networks considered seem to be partitioned into a well-defined region of low loss value and convex contours, surrounded by a well-defined region of high loss value and non-convex contours. This partitioning of chaotic and convex regions may explain the importance of good initialization strategies, and also the easy training behavior of "good" architectures. When using normalized random initialization strategies such as those proposed by Glorot and Bengio [11], typical neural networks attain an initial loss value less than 2.5. The well behaved loss landscapes in Figure 5 (ResNets, and shallow VGG-like nets) are dominated by large, flat, nearly convex attractors that rise to a loss value of 4 or greater. For such landscapes, a random initialization will likely lie in the "well- behaved" loss region, and the optimization algorithm might never "see" the pathological non-convexities that occur on the high-loss chaotic plateaus. Chaotic loss landscapes (ResNet-56/110-noshort) have shallower regions of convexity that rise to lower loss values. For sufficiently deep networks with shallow enough attractors, the initial iterate will likely lie in the chaotic region where the gradients are uninformative. In this case, the gradients "shatter" [1], and training is impossible. SGD was unable to train a 156 layer network without skip connections (even with very low learning rates), which adds weight to this hypothesis.

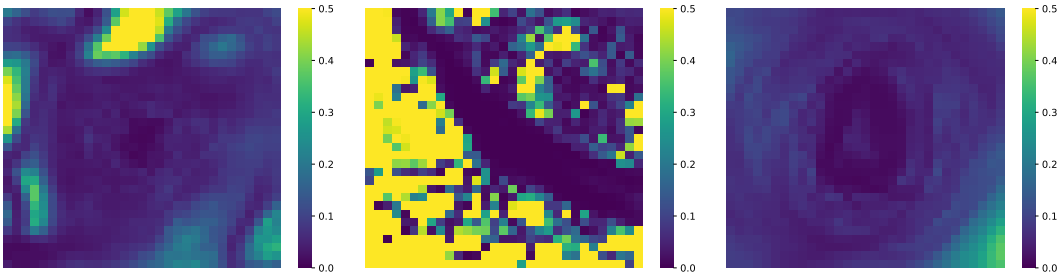
Landscape Geometry Affects Generalization Both Figures 5 and 6 show that landscape geometry has a dramatic effect on generalization. First, note that visually flatter minimizers consistently correspond to lower test error, which further strengthens our assertion that filter normalization is a natural way to visualize loss function geometry. Second, we notice that chaotic landscapes (deep networks without skip connections) result in worse training and test error, while more convex landscapes have lower error values. In fact, the most convex landscapes (Wide-ResNets in the top row of Figure 6), generalize the best of all, and show no noticeable chaotic behavior.

A note of caution: Are we really seeing convexity? We are viewing the loss surface under a dramatic dimensionality reduction, and we need to be careful how we interpret these plots. One way to measure the level of convexity in a loss function is to compute the *principle curvatures*, which are simply eigenvalues of the Hessian. A truly convex function has non-negative curvatures (a positive

semi-definite Hessian), while a non-convex function has negative curvatures. It can be shown that the principle curvatures of a dimensionality reduced plot (with random Gaussian directions) are weighted averages of the principle curvatures of the full-dimensional surface (where the weights are Chi-square random variables).

This has several consequences. First of all, if non-convexity is present in the dimensionality reduced plot, then non-convexity must be present in the full-dimensional surface as well. However, apparent convexity in the low-dimensional surface does not mean the high-dimensional function is truly convex. Rather it means that the positive curvatures are dominant (more formally, the *mean* curvature, or average eigenvalue, is positive).

While this analysis is reassuring, one may still wonder if there is significant "hidden" non-convexity that these visualizations fail to capture. To answer this question, we calculate the *minimum* and *maximum* eigenvalues of the Hessian, λ_{min} and λ_{max}.[4] Figure 7 maps the ratio $|\lambda_{min}/\lambda_{max}|$ across the loss surfaces studied above (using the same minimizer and the same random directions). Blue color indicates a more convex region (near-zero negative eigenvalues relative to the positive eigenvalues), while yellow indicates significant levels of negative curvature. We see that the convex-looking regions in our surface plots do indeed correspond to regions with insignificant negative eigenvalues (i.e., there are not major non-convex features that the plot missed), while chaotic regions contain large negative curvatures. For convex-looking surfaces like DenseNet, the negative eigenvalues remain extremely small (less than 1% the size of the positive curvatures) over a large region of the plot.



(a) Resnet-56 (b) Resnet-56-noshort (c) DenseNet-121

Figure 7: For each point in the filter-normalized surface plots, we calculate the maximum and minimum eigenvalue of the Hessian, and map the ratio of these two.

7 Visualizing Optimization Paths

Finally, we explore methods for visualizing the trajectories of different optimizers. For this application, random directions are ineffective. We will provide a theoretical explanation for why random directions fail, and explore methods for effectively plotting trajectories on top of loss function contours.

Several authors have observed that random directions fail to capture the variation in optimization trajectories, including [10, 29, 28, 27]. Example failed visualizations are depicted in Figure 8. In Figure 8(a), we see the iterates of SGD projected onto the plane defined by two random directions. Almost none of the motion is captured (notice the super-zoomed-in axes and the seemingly random walk). This problem was noticed by [13], who then visualized trajectories using one direction that points from initialization to solution, and one random direction. This approach is shown in Figure 8(b). As seen in Figure 8(c), the random axis captures almost no variation, leading to the (misleading) appearance of a straight line path.

7.1 Why Random Directions Fail: Low-Dimensional Optimization Trajectories

It is well-known that two random vectors in a high dimensional space will be nearly orthogonal with high probability. In fact, the expected cosine similarity between Gaussian random vectors in n dimensions is roughly $\sqrt{2/(\pi n)}$ ([12], Lemma 5).

[4] We compute these using an implicitly restarted Lanczos method that requires only Hessian-vector products (which are calculated directly using automatic differentiation), and does not require an explicit representation of the Hessian or its factorization.